

Todo crédito é uma decisão sob incerteza

Errar para os dois lados custa dinheiro. A questão é escolher a quem emprestar antes de saber quem vai pagar.

Aprovar um mau pagador

A perda aparece no balanço: o dinheiro emprestado não volta.

Recusar um bom pagador

A perda é silenciosa: ninguém contabiliza a receita que deixou de entrar.

8,07%
inadimplência real na base
(TARGET = 1)

OBJETIVO

Estimar a **probabilidade de default (PD)** de cada cliente e transformar esse número numa decisão de concessão.

COMO SABEMOS SE DEU CERTO

AUC-ROC · a métrica técnica

Redução da inadimplência

Resultado líquido em R\$

O que os dados revelam

Antes de modelar: entender o desbalanceamento, os valores sujos e o que de fato prevê risco.

1

Base fortemente desbalanceada

Só 8,07% são inadimplentes — por isso a métrica-guia é o AUC, e não a acurácia.

2

Valores sentinela tratados

DAYS_EMPLOYED = 365243 era o código para “sem emprego”. Limpo antes de treinar.

3

Muitos nulos com significado

Em bureau e aplicações anteriores, o vazio quer dizer algo: o cliente não tem histórico.

4

Os preditores mais fortes

EXT_SOURCE_1/2/3 (scores externos), idade e tempo de emprego.

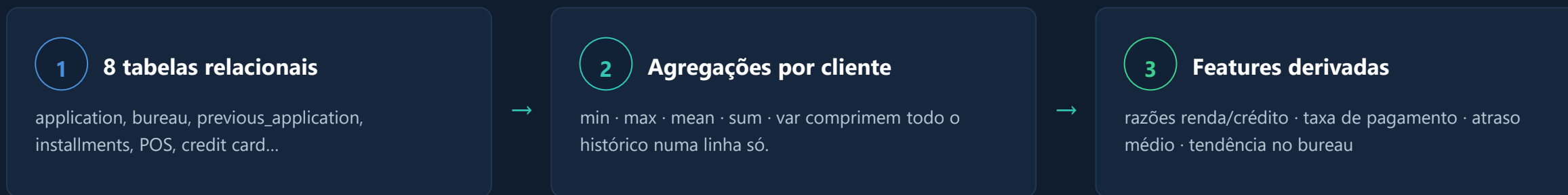
DISTRIBUIÇÃO DO TARGET



Adimplentes 91,93% Inadimplentes 8,07%

De 8 tabelas a uma linha por cliente

Um cliente tem dezenas de linhas de histórico espalhadas. O modelo precisa de exatamente uma decisão por cliente.



836

features entram no modelo

A ABT final tem 838 colunas: as 836 features, mais SK_ID_CURR e TARGET. **Uma tabela única por cliente** — pronta para treino, com o risco relativo já capturado por razões e agregações do histórico.

Só 249 dessas features vêm do formulário de proposta. As outras 587 vêm do que o cliente já deixou registrado: bureau, cartão, parcelas e aplicações anteriores.

Do baseline ao LightGBM

Só vale adotar o modelo complexo se o ganho justificar — e com o overfitting sob controle.

ROC-AUC (out-of-fold)

Regressão Logística

0,7730



LightGBM

0,7909



▲ +1,8 p.p. de AUC — o ganho justifica a complexidade

CONTROLE DE OVERFITTING

✓ K-Fold estratificado (5)

✓ Early stopping

✓ Regularização L1 / L2

✓ Árvores rasas · max_depth = 4

TUNING · OPTUNA

num_leaves = 18 · max_depth = 4 · learning_rate = 0,025 · n_estimators = 2319 · min_child_samples = 149

A diferença entre 0,7730 e 0,7909 parece pequena no papel. **Numa carteira de 300 mil contratos, ela é a fronteira entre aprovar e recusar milhares de clientes.**

Dá para confiar na decisão

Métricas sem vazamento, folds consistentes e explicação individual de cada crédito concedido ou negado.

0,7909

AUC-ROC out-of-fold
cada cliente pontuado sem se ver no
treino

0,4424

KS
separação entre bons e maus
pagadores

0,5818

Gini
($2 \times \text{AUC} - 1$)

SHAP

explica cada decisão
"por que este cliente foi recusado?"

Drivers de risco · importância média entre os 5 folds



POLÍTICA DE DECISÃO

PD < 8%

aprovar automaticamente

8% – 30%

análise manual (zona incerta)

> 30%

recusar

KPIs de negócio: corte de recusa · inadimplência evitada · resultado líquido em R\$

Como a aplicação está montada

Todo o caminho, da tabela bruta até a tela do analista — com quem dispara cada etapa, onde os dados ficam gravados e quem lê de onde.



Onde tudo fica gravado

PostgreSQL 16, quatro schemas. Cada um responde a uma pergunta diferente sobre a operação.

| **staging** clean_data

Os dados já limpos, antes de virarem feature. A camada de conferência.

| **feature_store** abt

Uma linha por cliente, 836 features. É daqui que a API lê a cada decisão.

| **serving** predictions

Toda decisão tomada fica registrada: entrada, PD, faixa, versão do modelo, latência.

| **mlops** model_registry · monitoring_runs · drift_metrics · performance_metrics

Qual modelo está no ar e como ele vem se comportando ao longo do tempo.

| **Nada é recalculado na hora da decisão.** A API faz uma leitura por chave primária — por isso responde em milissegundos mesmo com 4 GB de histórico atrás dela.

356.251

clientes na feature store

4,0 GB

de features gravadas

1,9 ms

para ler um cliente inteiro

DECISÃO DE PROJETO

As 836 features moram num JSONB — e as variáveis de negócio são colunas geradas a partir dele.

De onde vêm as outras variáveis?

O formulário da proposta preenche só uma parte. O resto o cliente já deixou registrado — e é justamente o resto que muda a decisão.

As 836 features, por origem



- aplicações anteriores 260
- cartão de crédito 142
- parcelas pagas 29
- cadastro e proposta 249
- bureau de crédito 135
- POS / consignado 21

3 variáveis digitadas na tela

PD 4,6%

o que faltava virou valor médio

APROVAR

658 variáveis buscadas no banco

PD 34,6%

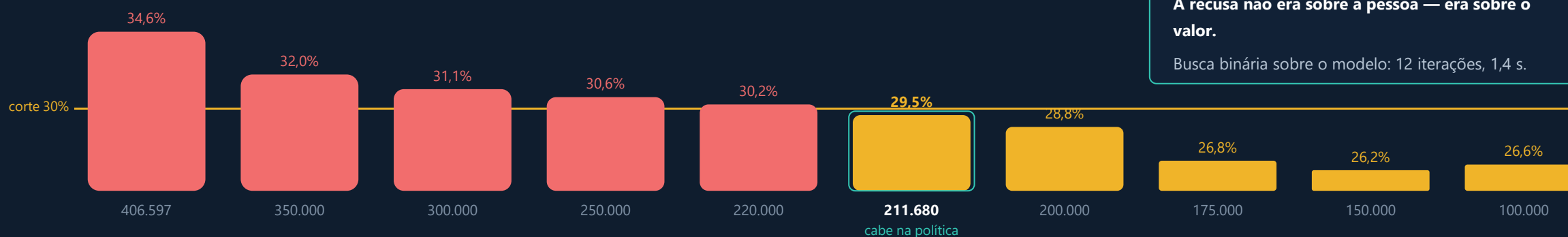
o histórico completo inverteu a decisão

RECUSAR

E SE ELE PEDISSE MENOS?

Variando só o valor do crédito. As outras 655 features continuam vindo da feature store a cada uma das 22 chamadas.

Desfecho real do cliente 100002 na base: **inadimplente**.



A recusa não era sobre a pessoa — era sobre o valor.

Busca binária sobre o modelo: 12 iterações, 1,4 s.

Onde entra o Airflow

Não é detalhe de infraestrutura. É o que impede a feature store de envelhecer e o modelo de decidir sobre o cliente de seis meses atrás.

credit_risk_ingestion

@daily

sanitize → build_abt → load_feature_store

Mantém a feature store viva. Carga por UPSERT: atualiza quem mudou, não apaga ninguém.

credit_risk_training

manual

train → register_model

Retreina e publica a nova versão no registry. Disparo manual, porque trocar modelo é decisão de negócio.

credit_risk_batch_scoring

@daily

pontua a carteira inteira

2.000 clientes por segundo. Grava tudo em serving.predictions, com a versão do modelo junto.

credit_risk_monitoring

@daily

data drift || prediction drift || performance

Três tarefas em paralelo. É a DAG que avisa quando alguma coisa saiu do lugar.

Metastore em PostgreSQL · LocalExecutor · as DAGs sobem no mesmo docker compose up do resto.

A DAG de ingestão tem uma trava: ela recusa publicar uma ABT construída sobre amostra, porque isso apagaria o histórico de quem já está no banco.

Saber que o modelo piorou antes do prejuízo

Três camadas. As duas primeiras avisam cedo, olhando só a entrada; a terceira confirma — mas só quando o desfecho do crédito chega.

Drift de dados

6 features

em nível severo

PSI por feature entre a população de treino e a que está chegando. Limiares: 0,10 moderado · 0,25 severo.

Drift de predição

PSI 0,243

PD média 0,080 → 0,149

A carteira que chega ficou mais arriscada — e isso aparece antes de qualquer contrato vencer.

Performance por safra

0,713

abaixo do piso de 0,751

AUC das últimas safras: 0,776 · 0,763 · 0,819 · 0,713.
Tolerância de 5% sobre o AUC de referência → alerta.

O ACHADO QUE NÃO FOI SIMULAÇÃO

PAYMENT_RATE apareceu com **PSI 1,51** entre o treino e a população nova. Fomos investigar: no treino, 12% dos clientes tinham crédito de prazo muito longo — mediana de 34 anos e R\$ 1,08 milhão. Na população nova esse segmento simplesmente não existe mais: zero clientes, prazo mediano caiu de 20 para 16 anos.

Um produto inteiro saiu da carteira — e o monitoramento apontou sozinho, sem ninguém procurar.

O que fazer com a decisão depois de tomada

PROPOSTA

Entre a PD e o efeito no negócio ainda tem uma pessoa abrindo outro sistema. É esse intervalo que a automação fecharia.

PD < 8%

APROVAR

Liberar no sistema de originação · avisar o cliente · registrar a decisão com a versão do modelo.

8% – 30%

ANÁLISE MANUAL

Abrir ticket para a mesa já com a explicação SHAP anexada · priorizar a fila por valor e proximidade do corte.

> 30%

RECUSAR

Antes de comunicar, procurar uma oferta alternativa viável — quase sempre a recusa é sobre o valor pedido, não sobre a pessoa.

TRÊS AGENTES DE IA PROPOSTOS

Explicação

“por que este cliente foi recusado?”

Recebe as contribuições SHAP que a API já calcula e escreve a justificativa em linguagem de negócio.

Retenção

“existe crédito que ele conseguiria pagar?”

Interpreta a busca por oferta alternativa e redige a contraproposta. A conta dura é do modelo; o agente comunica.

Monitoramento

“o que aconteceu e o que eu faço?”

Transforma a tabela de PSI e AUC no parágrafo que alguém de plantão precisa entender às 3 da manhã.

Já está de pé: SHAP exposto por API · simulação que recalcula as 9 features derivadas · log auditável de toda decisão · registry do modelo · Airflow.

Falta o motor de ações — deixado como proposta de propósito: sem um sistema de originação real para receber a oferta, seria uma tabela que ninguém consome.